



Попов Александр.pdf

PDF



подробное_описание_работы_и_...

Document

в этих документах мои основные достижения и заслуги. я хочу сделать сайт со своим портфолио, чтобы каждый мог посмотреть, что у меня есть, какие кейсы я решал, с какими командами работал, какие дашборды и для чего я делал, какую ценность они принесли, какая архитектура и техстек присутствовали в компании, какие инструменты и решения были приняты и так далее. все то, что в резюме упоминается, но очень коротко и порою может звучать совсем не впечатляюще. я как-то видел у одного кандидата, которого я же и отбирал, сайт на github, где было подробно расписано его резюме, были примеры дашбордов со скриншотами. я хочу, чтобы ты проанализировал мои вводные, мою цель, скорректировал и подсказал где это реализовать, как это реализовать, может составил некую структуру, что стоит включить туда, нужно ли разбивать портфолио на марс и атом или сразу сделать общее представление на основе всего опыта который есть (наверное так логичнее, актуальное состояние, да и с предыдущего места у меня не сохранилось примеров дашбордов, хотя есть часть классных решений – ты это увидишь в описании работы). ну и можешь дать оценку, насколько вообще это перспективное направление для меня, для большего внимания к моей кандидатуре?

Сначала разберу твои материалы и затем сопоставлю их с 2–3 реалистичными форматами портфолио-сайта. Дальше дам не абстрактные советы, а конкретную структуру: что показывать, что не показывать, как упаковать Atom и Mars, и насколько это реально усилит твою кандидатуру.

У тебя уже вырисовывается не просто “резюме на сайте”, а сильный кейсовый профиль руководителя BI/analytics. По материалам видно, что твоя ценность не в

одном инструменте, а в связке: запуск функции с нуля, построение delivery-процесса, влияние на топов, кросс-функциональная работа и измеримый эффект.

Thought for 2m 21s >

Да, это перспективная идея. Но тебе нужен не просто “сайт-резюме”, а **портфолио BI/Analytics leader в формате case-based profile**.

По твоим материалам видно, что у тебя уже есть содержимое для сильного сайта: в Atom ты не просто делал отчёты, а запускал аналитическое направление с нуля, выстроил delivery-процесс, собрал команду, довёл платформу до 25+ источников, 100+ витрин, 100+ DAGs и 50+ дашбордов, а также привёл использование BI от 0 до 200 MAU и эффект до 1000+ часов автоматизации в месяц и ~300 млн рублей экономии в год. В Mars у тебя есть очень сильные коммерческие кейсы с измеримым эффектом: +59 млн на расширении полки, ещё +200 млн после масштабирования практики на X5 и около +600 млн через управление ДМП. Это как раз тот материал, который на обычном резюме не раскрывается, а на сайте начинает работать.

📄 Попов Александр

📄 подробное_описание_работы_и_дос...

📄 подробное_описание_работы_и_дос...

Главный вывод

Для тебя сайт — это **не замена резюме**, а усилитель доверия на этапе, когда рекрутер, hiring manager или будущий руководитель уже заинтересовались профилем и хотят понять масштаб. Для lead/head ролей это особенно полезно, потому что твоя ценность лежит сразу в нескольких слоях:

- бизнес-эффект,
- управленческий масштаб,
- зрелость BI-процесса,
- архитектурное мышление,
- умение показывать данные как инструмент принятия решений.

Именно это плохо помещается в 1–2 страницы CV, но хорошо раскрывается в кейсах.

Что я бы скорректировал в самой идее

Не делай сайт в логике “**подробно пересказал всё резюме**”.

Это слабый формат.

Делай сайт в логике:

Кто я → какой масштаб задач решал → 5–7 отобранных кейсов → как я строю BI-функцию → стек/архитектура → выступления/материалы → контакты.

То есть не хронологический пересказ карьеры, а **упаковка профессиональной ценности**.

Разбивать ли на Mars и Atom

Полностью разделять сайт на "Mars" и "Atom" не стоит.

Лучше сделать **единый сайт**, а внутри — кейсы с тегами:

- Atom
- Mars
- Sales Analytics
- BI Strategy
- Data Platform
- Telematics
- Finance
- Data Quality
- Team Leadership

Почему так лучше

Mars у тебя даёт очень сильный **денежный эффект**.

Atom даёт очень сильный **масштаб, зрелость процесса и лидерство**.

Вместе они показывают гораздо более цельный профиль: ты не только умеешь "считать", но и умеешь строить функцию, процессы и систему принятия решений.

 Попов Александр

 подробное_описание_работы_и_дос...

То есть логика должна быть такой:

- **главная страница** — общее позиционирование,
- **кейсы** — вперемешку, но с фильтрами или тегами,
- **отдельные страницы компаний** — не обязательны.

Где это реализовать

Я бы рассматривал для тебя 3 варианта.

1. Лучший баланс: GitHub + Next.js + Vercel + свой домен

Это мой основной рекомендационный вариант.

Почему:

- Vercel поддерживает автодеплой из Git-репозитория и custom domains.

Vercel +2

- У Vercel есть готовые portfolio starter kits на Next.js/Markdown, то есть не нужно поднимать всё с нуля. Vercel +3
- Такой формат для тебя органичен: он выглядит современно, даёт хороший контроль над структурой, и сам факт Git-репозитория и аккуратной реализации добавляет технической убедительности.

Когда выбирать:

если хочешь сделать один раз нормально и потом спокойно поддерживать.

2. Самый быстрый старт: Framer + свой домен

Framer позволяет быстро опубликовать сайт, даёт свой домен вида `framer.app`, поддерживает публикацию и подключение custom domain, а также имеет большой маркетплейс шаблонов. Framer +3

Плюс: быстро, красиво, почти без кода.

Минус: сигнал о технической глубине слабее, чем у GitHub/Next.js.

Когда выбирать:

если твоя цель — выйти в публичный вид за 2–3 вечера без фронтенд-возни.

3. Самый простой статический вариант: GitHub Pages или Cloudflare Pages

GitHub Pages — это статический хостинг для HTML/CSS/JS прямо из GitHub-репозитория; он поддерживает custom domains. GitHub Docs +3

Cloudflare Pages тоже подключается к GitHub/GitLab и поддерживает автоматические деплои и custom domains. Cloudflare Docs +3

Когда выбирать:

если нужен минималистичный и очень недорогой вариант без лишней сложности.

Что я бы рекомендовал именно тебе

Основной выбор: Next.js на Vercel.

Резервный быстрый выбор: Framer.

Для твоего профиля важно, чтобы сайт выглядел не как “визитка дизайнера”, а как **структурированная профессиональная система кейсов**. Поэтому для тебя особенно хорошо подходит формат:

- строгий чистый дизайн,

- карточки кейсов,
- отдельные case study pages,
- блок с архитектурой,
- блок с leadership/process.

Какой должна быть структура сайта

1. Home

Первый экран:

Александр Попов

BI & Analytics Lead

Строю аналитические функции, BI-процессы и управленческую отчётность, которые помогают бизнесу принимать решения быстрее и точнее.

Ниже — 5–7 сильных метрик. Например:

- 6+ лет в BI и аналитике
- 50+ дашбордов
- 25+ источников данных
- 100+ витрин данных
- 100+ Airflow DAGs
- 30+ команд вовлечены в работу с данными
- 200 MAU BI-системы
- 1000+ часов/мес автоматизации
- ~300 млн ₽ экономии в год
- +59 млн / +200 млн / +600 млн в коммерческих кейсах Mars

Эти цифры у тебя уже есть в материалах.  Попов Александр

 подробное_описание_работы_и_дос...

 подробное_описание_работы_и_дос...

2. About / Positioning

Не биография на пол-экрана, а короткий блок:

- чем занимаешься,
- на каком уровне работаешь,
- как выглядишь как leader,
- в чём твоя специализация.

Тезисно:

“Я руковожу BI/analytics-направлением на стыке бизнеса, процессов и данных. Запускаю и масштабирую аналитические решения, строю delivery-модель, развиваю команды и превращаю данные в управленческие решения.”

3. Selected Cases

Это центр всего сайта.

Не меньше 5 и не больше 8 кейсов на старте.

Я бы взял такие:

Atom

1. Запуск аналитической функции с нуля
2. BI delivery-процесс и operating model команды
3. Telematics / fleet utilization dashboard
4. Executive reporting for finance / operations / risk
5. Self-service и популяризация data culture

Mars

6. Модель расширения полочного пространства в Пятёрочке / X5
7. Контроль ДМП и рост покрытия до 100%
8. Sales incentive / KPI analytics / national reporting
(этот можно оставить как дополнительный, не обязательно в MVP)

Как оформлять каждый кейс

У тебя не должно быть просто “описания проекта”.

Должен быть один стандартный шаблон.

Шаблон case page

1. Контекст
2. Проблема бизнеса
3. Моя роль
4. Команда и взаимодействие
5. Что было сделано
6. Архитектура / data flow / стек
7. Что видел пользователь или руководитель
8. Результат
9. Что изменилось после внедрения
10. Ограничения / confidentiality note

Очень важно: отдельно писать

- что делал лично ты,
- что делала твоя команда,
- что делали смежники.

Для lead/head позиций это критично. Иначе кейсы выглядят либо размыто, либо как присвоение всей работы себе.

Как показывать Atom, если есть ограничения по NDA

Публиковать реальные внутренние данные, названия чувствительных сущностей, схемы таблиц и слишком детальные внутренние скриншоты не надо.

Вместо этого:

- показывать **санитизированные скриншоты**,
- обрезать чувствительные поля,
- размывать цифры, где нужно,
- рисовать **упрощённые архитектурные схемы**,
- использовать **reconstructed mockups**, если исходник нельзя публиковать.

Для Mars, где у тебя нет части скриншотов, это вообще нормальный путь: делать не “реальный экран”, а **схему решения**:

- входные данные,
- логика сегментации,
- управленческое решение,
- результат в деньгах.

Это абсолютно рабочий формат.

Что ещё обязательно включить

Operating model / How I build BI

Это твой очень сильный дифференциатор.

Отдельный раздел, где ты покажешь:

- как собираешь требования,
- как приоритизируешь,
- как устроен путь от запроса до релиза,
- как устроены роли SA / DA / DE,
- как выглядит S2T / витрина / дашборд / тест / prod,
- как обеспечивается качество данных,
- как ты ускоряешь time-to-market.

По материалам это у тебя одна из самых сильных сторон.

 Попов Александр

 подробное_описание_работы_и_дос...

Tech stack & architecture

Не простая россыпь логотипов, а понятная схема:

- источники,
- ingestion,
- storage,
- modeling,
- orchestration,
- BI layer,
- governance / documentation,
- workflow tools.

У тебя для этого уже есть достаточный материал: DataLens, Superset, Trino, S3, Iceberg, Airflow, PySpark, Jira, Confluence, GitLab, Dremio, Power BI, Excel и т.д.

 Попов Александр

 Попов Александр

Talks / Public activity

Сюда уже можно вынести выступление на конференции Яндекса и внутренние воркшопы. Но не ставь это главным достижением на сайте. По твоим материалам публичная активность — это **усилитель доверия**, а не ядро ценности. Ядро — это кейсы, масштаб и влияние.

 подробное_описание_работы_и_дос...

 подробное_описание_работы_и_дос...

Resume / Contact

Обязательно:

- кнопка скачать CV в PDF,
- LinkedIn / Telegram / email / GitHub,
- короткая форма или простой СТА.

Что не стоит делать

1. Не делать слишком длинную автобиографию

Твой текущий материал очень богатый, но в сыром виде сайт получится перегруженным.

2. Не делать сайт “по работодателям”

Это создаёт ощущение архива, а не сильного позиционирования.

3. Не выкладывать всё подряд

Лучше 6 сильных кейсов, чем 20 слабых карточек.

4. Не злоупотреблять перечислением инструментов

Инструменты сами по себе никого не впечатляют. Впечатляет связка:
проблема → решение → результат → твоя роль.

5. Не оставлять кейсы без визуала

Даже если нет настоящего дашборда, нужен хотя бы один из вариантов:

- скриншот,
- mockup,
- архитектурная схема,
- before/after,
- flowchart.

Насколько это поможет твоей кандидатуре

Моя оценка: **да, поможет заметно**, но только при правильной упаковке.

Где это даст наибольший эффект

- lead/head роли в BI, analytics, data,
- компании, где важно влияние на бизнес, а не только SQL,
- ситуации, когда тебя смотрит hiring manager или CDO/Head of Data,
- интервью, где нужно быстро показать глубину и масштаб.

Где эффект будет слабее

- массовый отклик через ATS,
- вакансии, где до человека почти не доходят внешние ссылки,
- процессы, где нужен только формальный matching по стеку.

То есть сайт не заменит хорошее CV, но **сильно повышает доверие после первого интереса**. Для твоего профиля это особенно ценно, потому что у тебя много того, что трудно доказать одной строкой в резюме.

Самый рациональный вариант запуска

Не пытайся сразу сделать идеальный сайт на 20 страниц.

Сделай **MVP из 6 страниц**:

1. Home
2. About
3. Cases
4. Operating Model
5. Talks / Resume
6. Contact

И 5–6 кейсов внутри.

Каким должен быть MVP-контент именно у тебя

Я бы начал с такого набора:

- **Home** — позиционирование + метрики
- **Case 1** — Atom: запуск аналитики с нуля
- **Case 2** — Atom: BI delivery process
- **Case 3** — Atom: telematics/fleet dashboard
- **Case 4** — Mars: shelf expansion model
- **Case 5** — Mars: DMP coverage / 600M impact
- **Case 6** — Operating model / architecture / self-service

Этого уже достаточно, чтобы сайт выглядел серьёзно.

Итоговая рекомендация

Тебе стоит делать такой сайт. Но делать его надо не как “расширенное резюме”, а как **упакованное портфолио руководителя BI и аналитики с кейсами, архитектурой и управленческим подходом.**

Самая правильная для тебя формула сейчас:

единый сайт + лучшие кейсы из Mars и Atom + сильный акцент на роль, масштаб и эффект + аккуратные визуалы + свой домен + чистая строгая структура.

Рациональный следующий шаг — сначала собрать MVP на 5–6 кейсов и только потом расширять его, а не пытаться описать весь опыт сразу.

Sources

     ...  Sources

в общем то мне кажется, что github pages наверное наиболее понятный вариант, напиши инструкцию что сделать, чтобы там все реализовать, или может можно сделать через codex? если да, то напиши инструкцию для него, чтобы он понял что и как создавать и сделал всю нужную структуру для моего сайта

Проверю актуальные варианты по GitHub Pages и по Codex, чтобы инструкция была не “по памяти”, а рабочая на сейчас. Затем дам тебе два готовых сценария: руками через GitHub Pages и через Codex с чётким промптом/ТЗ.

Thought for 37s >

Да — **через GitHub Pages** это вполне логичный вариант, а через **Codex** можно сильно ускорить старт: Codex умеет работать с кодовой базой в репозитории, а GitHub Pages публикует статический сайт напрямую из файлов `HTML/CSS/JS` в репозитории или из выбранной ветки/папки. Для простого портфолио это, по сути, самый прямой маршрут: без сервера, без сложного деплоя, с понятной структурой и возможностью позже подключить свой домен. GitHub Docs +3

Что я рекомендую тебе выбрать

Для твоего кейса я бы брал такой стек:

- **GitHub Pages**
- **обычный статический сайт:** `index.html + css + js`
- **без Jekyll на старте**
- **Codex** для генерации структуры, секций, стилей и шаблона кейсов

Почему так:

- быстрее запуск;
- меньше точек отказа;
- проще править руками;
- сайт будет достаточно “профессиональным”, если грамотно собрать контент;
- позже можно без боли перейти на более сложный стек.

Jekyll GitHub Pages поддерживает нативно, но он тебе сейчас не обязателен; для первого MVP это только добавит лишний слой абстракции. GitHub Pages в базовом варианте может публиковать просто статические файлы из репозитория.

GitHub Docs +3

Вариант, который я считаю оптимальным

Сценарий

1. Создаёшь репозиторий на GitHub
2. Включаешь GitHub Pages
3. Даёшь Codex задачу сгенерировать структуру сайта
4. Коммитишь результат
5. Публикуешь сайт

Как сделать это руками через GitHub Pages

Шаг 1. Создай репозиторий

Назови его так:

- `portfolio`
или
- `aleksandr-popov-portfolio`

Если захочешь сайт на стандартном GitHub Pages URL вида `username.github.io`, тогда имя репозитория должно быть именно `username.github.io`. GitHub Pages также умеет публиковать сайт из обычного репозитория, если выбрать источник публикации. [GitHub Docs +2](#)

Шаг 2. Подготовь минимальную структуру

Создай в репозитории такие файлы:

```
/
  index.html
  /assets
    /css
      styles.css
    /js
      main.js
    /img
      avatar.jpg
      preview-case-1.png
      preview-case-2.png
  /pages
    about.html
    cases.html
    case-atom-analytics-function.html
    case-atom-telematics-dashboard.html
    case-mars-shelf-expansion.html
    operating-model.html
    contact.html
  404.html
  README.md
```

Шаг 3. Включи GitHub Pages

В GitHub:

- открой репозиторий
- зайди в **Settings**
- открой раздел **Pages**
- в **Build and deployment** выбери публикацию из branch
- выбери ветку `main`
- папку `/root`

GitHub Pages позволяет публиковать сайт либо из корня ветки, либо из папки

`/docs` на выбранной ветке. Для простоты бери корень репозитория. [GitHub Docs](#)

Шаг 4. Сделай первый коммит

Залей хотя бы простой `index.html`, например:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Александр Попов – BI & Analytics Portfolio</title>
  <link rel="stylesheet" href="assets/css/styles.css" />
</head>
<body>
  <header>
    <h1>Александр Попов</h1>
    <p>BI & Analytics Lead</p>
  </header>

  <main>
    <section>
      <h2>Портфолио</h2>
      <p>Сайт в разработке. Здесь будут кейсы, архитектура, дашборды и ре:
    </section>
  </main>

  <script src="assets/js/main.js"></script>
</body>
</html>
```

После пуша GitHub Pages обычно публикует сайт автоматически из выбранного источника. [GitHub Docs +1](#)

Шаг 5. Проверь URL

Сайт будет доступен либо по адресу вида:

- `https://username.github.io`
- или
- `https://username.github.io/repository-name/`

Это зависит от типа Pages-сайта и имени репозитория. В настройках Pages GitHub показывает итоговый URL публикации. [GitHub Docs +1](#)

Шаг 6. Когда захочешь — подключи свой домен

GitHub Pages поддерживает custom domain. Для этого нужно:

- указать домен в настройках Pages,
- настроить DNS-записи у регистратора,
- при необходимости пройти verify domain. [GitHub Docs +2](#)

Можно ли сделать через Codex

Да. Это как раз хороший вариант.

Есть несколько способов работы с Codex:

- **Codex web** — подключаешь GitHub, и он работает с репозиториями и может предлагать изменения через PR;
- **Codex CLI** — работает локально в твоей директории;
- **IDE extension** — если хочешь править из редактора. Codex доступен в рамках ChatGPT Plus/Pro/Business/Edu/Enterprise, а в web-версии его можно подключить к GitHub-репозиториям. [OpenAI Developers +3](#)

Для тебя я бы выбрал **Codex web + GitHub repo** или **Codex CLI**, если тебе комфортнее локально.

Какой сценарий с Codex я рекомендую

Самый практичный

1. Создай пустой GitHub-репозиторий
2. Добавь туда `README.md` и `AGENTS.md`
3. Подключи репозиторий к Codex
4. Дай Codex одно большое ТЗ на создание структуры сайта
5. Пусть он создаст страницы, стили, навигацию и шаблоны кейсов
6. Затем уже по очереди проси его наполнять контентом

Что положить в AGENTS.md

Создай в корне репозитория файл `AGENTS.md` с таким содержимым:

↗ Markdown



AGENTS.md

Project

This repository contains a personal portfolio website for Aleksandr Popov,

Goal

Build a clean, professional, static portfolio website optimized for GitHub. No frameworks are required for the initial version unless explicitly requested. Use plain HTML, CSS, and a small amount of JavaScript.

Website goals

The site should present:

- professional positioning
- selected case studies
- dashboards and screenshots
- architecture and tech stack
- BI operating model and delivery process
- measurable business impact
- contact information
- downloadable resume

Constraints

- Keep the site static and GitHub Pages-friendly.
- Do not use backend services.
- Avoid heavy frontend frameworks.
- Use semantic HTML.
- Use clean, restrained visual design.
- The style should look executive and product-minded, not flashy.
- The website language should be Russian by default.
- Structure content so it can later be translated into English.

Required pages

- Home
- About
- Cases
- Individual case pages
- Operating Model / How I build BI
- Tech Stack / Architecture
- Contact

UX requirements

- Desktop-first but responsive on mobile.

- Sticky header navigation.
- Cards for cases.
- Reusable case study template.
- Image gallery/lightbox optional.
- Clear CTA to contact / download CV.

Content model for each case

Each case page should contain:

1. Context
2. Business problem
3. My role
4. Team and stakeholders
5. Solution
6. Architecture / data flow
7. Dashboard / output
8. Result / impact
9. Tools used
10. Confidentiality note if needed

Technical requirements

- Organize CSS and JS into assets folders.
- Use relative links compatible with GitHub Pages.
- Add 404.html.
- Add README with deployment instructions.
- Keep code simple and maintainable.
- Prefer reusable CSS classes over duplicated styles.

Important

Do not invent fake achievements.

Where content is missing, create placeholders clearly marked as TODO.

Промпт для Codex, который можно просто вставить

Ниже — версия, которую я бы реально дал Codex.

⟨⟩ Plain text



Create a complete static portfolio website for GitHub Pages.

Project context:

This is a personal portfolio website for Aleksandr Popov, a BI & Analytics

Target audience:

- recruiters
- hiring managers
- heads of data / CDOs
- executives who want to assess business impact and leadership scale

Main positioning:

Aleksandr is not just a dashboard developer. He builds analytics functions

Requirements:

1. Build the site as a static website fully compatible with GitHub Pages.
2. Use plain HTML, CSS, and minimal JavaScript.
3. Use semantic markup and clean file structure.
4. Language: Russian.
5. Design: clean, restrained, executive, modern, not flashy.
6. Responsive layout.
7. Add reusable components or patterns through plain HTML/CSS structure.

Create this structure:

- index.html
- pages/about.html
- pages/cases.html
- pages/operating-model.html
- pages/architecture.html
- pages/contact.html
- pages/case-atom-analytics-function.html
- pages/case-atom-telematics-dashboard.html
- pages/case-mars-shelf-expansion.html
- assets/css/styles.css
- assets/js/main.js
- assets/img/ placeholders if needed
- 404.html
- README.md

Content sections required:

Home page:

- hero section
- short positioning statement
- selected metrics / impact numbers cards
- selected cases preview
- tech stack preview
- CTA to cases and contact

About page:

- who Aleksandr is
- leadership focus
- approach to BI and analytics
- short career timeline

Cases page:

- grid of case cards
- tags / filters optional
- each card links to a case page

Case page template:

- title
- company / domain
- context
- business problem
- my role

- team / stakeholders
- what was built
- architecture / data flow block
- screenshots placeholder block
- business impact block
- tools / stack
- confidentiality note

Operating model page:

- how requests are collected
- prioritization
- roles in team
- delivery flow from business request to dashboard
- quality / governance principles

Architecture page:

- data sources
- storage / transformation / orchestration / BI layers
- tools and platforms
- simplified architecture diagram section placeholder

Contact page:

- email placeholder
- LinkedIn placeholder
- GitHub placeholder
- CV download placeholder

Design requirements:

- use a restrained color palette
- strong typography hierarchy
- cards, sections, anchors, sticky nav
- avoid excessive gradients or visual noise

Implementation details:

- all navigation should work with relative paths suitable for GitHub Pages
- add active nav state if practical
- add mobile menu if needed but keep it simple
- use TODO placeholders where real content is not available yet
- do not add fake company-confidential details

Also:

- write a README explaining how to publish this site on GitHub Pages
- include comments in HTML/CSS only where useful
- ensure the structure is easy to edit manually later

After creating the initial structure, summarize:

- what files were created
 - what content still needs to be filled manually
 - how to publish on GitHub Pages
-

Как работать с Codex по шагам

Вариант А — через Codex web

1. Создай репозиторий на GitHub
2. Подключи GitHub к Codex
3. Открой этот репозиторий в Codex
4. Добавь `AGENTS.md`
5. Вставь большой промпт выше
6. Попроси Codex создать весь scaffold сайта
7. Прими изменения и закоммить их в `main` или через PR

Codex web как раз рассчитан на работу с GitHub-репозиториями и может создавать изменения в кодовой базе. [OpenAI Developers +1](#)

Вариант В — через Codex CLI

Если хочешь локально:

1. создай папку проекта
2. инициализируй git
3. открой её в терминале
4. установи/запусти Codex CLI
5. дай тот же промпт
6. просмотри изменения
7. закоммить и пушни на GitHub

Codex CLI работает в выбранной локальной директории и может читать, менять и запускать код на машине. [OpenAI Developers](#)

Какой контент Codex стоит генерировать сразу, а какой нет

Сразу можно поручить Codex

- структуру страниц
- навигацию
- стиль
- шаблон карточек кейсов
- шаблон case study page
- блоки "metrics", "tech stack", "operating model"
- placeholders для скриншотов
- README

- 404 page

Не стоит поручать ему "с потолка"

- реальные достижения
- конкретные цифры
- формулировки твоих бизнес-эффектов без твоей проверки
- описание confidential кейсов без ручной редакции

Правильнее: сначала он делает каркас, потом ты по одной странице просишь его заполнять на основе твоих материалов.

Какой сайт сделать в первой версии

Я бы ограничил MVP так:

Обязательно

- Home
- About
- Cases
- 3 case pages
- Operating Model
- Architecture
- Contact

Первые 3 кейса

- Atom — запуск аналитической функции
- Atom — telematics / fleet utilization dashboard
- Mars — shelf expansion / commercial impact

Этого уже достаточно, чтобы сайт выглядел серьёзно.

Важный технический нюанс для GitHub Pages

Чтобы не ловить проблемы со ссылками:

- используй **относительные пути**
- не завязывайся на абсолютный `/`, если не уверен в формате URL
- проверь все ссылки между `index.html` и `/pages/...`
- изображения и стили тоже подключаай аккуратно через относительные пути

GitHub Pages публикует сайты либо как user/organization site, либо как project site, и у project site URL включает имя репозитория — из-за этого абсолютные пути часто ломаются. Общая логика публикации и выбор источника описаны в документации GitHub Pages. [GitHub Docs +2](#)

Моя практическая рекомендация

Сделай так:

Сегодня

- создай репозиторий
- добавь `AGENTS.md`
- подключи Codex
- дай ему промпт на scaffold

Потом

- покажи мне, что он сгенерировал
- и я помогу тебе:
 - допилить структуру,
 - переписать тексты под сильное позиционирование,
 - собрать первые кейсы,
 - сделать формулировки уровня hiring manager / CDO.

Если хочешь, следующим сообщением я могу сразу дать тебе **готовый** `README.md` + **готовый** `AGENTS.md` + **готовый стартовый** `index.html`, чтобы ты просто вставил их в репозиторий.

     ...  Sources